

運用 Spanner Graph 和 Spanner AI 功能打造即時零售推薦系統

來源：<https://codelabs.developers.google.com/spanner-ai-graph-recommendations?hl=zh-tw>

步驟數：13

在本程式碼研究室中，您將學習如何建構智慧型端對端產品推薦系統。您將使用 Google Cloud Spanner 的內建 AI 和圖形功能，從簡單的產品目錄建構出功能強大的推薦引擎，這類引擎可使用自然語言尋找產品、說明產品適合的原因，以及發掘相關項目。

目錄

1. 簡介
2. 事前準備
3. 架構總覽
4. 步驟 1：設定資料庫並提交第一項查詢。
5. 步驟 2：建立 AI 模型。
6. 步驟 3：產生及儲存向量嵌入
7. 步驟 4：建立 ANN 搜尋的向量索引
8. 步驟 5：使用 K 近鄰 (KNN) 搜尋功能尋找建議
9. 步驟 6：使用近似 (ANN) 搜尋功能尋找建議
10. 步驟 7：使用 LLM 解釋建議
11. 步驟 8：建立屬性圖
12. 步驟 9：結合向量搜尋和圖形查詢
13. 正在清除所用資源

1. 簡介

本程式碼研究室會引導您使用 [Spanner](#) 的 AI 和圖形功能，強化現有的零售資料庫。您將學到實用技巧，瞭解如何在 Spanner 中運用機器學習技術，為顧客提供更優質的服務。具體來說，我們會導入 k 近鄰 (kNN) 和近似近鄰 (ANN) 演算法，發掘符合個別消費者需求的新產品。您也會整合 LLM，以清楚的自然語言說明推薦特定產品的原因。

除了建議之外，我們還會深入探討 Spanner 的圖功能。您將根據顧客的購買記錄和產品說明，使用圖形查詢來模擬產品之間的關係。這項做法可發掘深層相關的項目，大幅提升「購買此商品的顧客也買了」或「相關項目」功能的關聯性和效果。完成本程式碼研究室後，您將具備相關技能，完全透過 Google Cloud Spanner 建構智慧型、可擴充且回應迅速的零售應用程式。

情境

你任職於電子設備零售商。您的電子商務網站具有標準的 Spanner 資料庫，其中包含 `Products`、`Orders` 和 `OrderItems`。

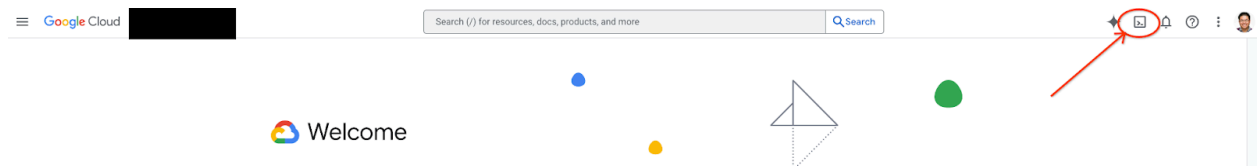
顧客造訪你的網站，並有特定需求：「我想購買高效能鍵盤。我偶爾會在海灘上寫程式，所以可能會弄濕。」

您的目標是使用 Spanner 的進階功能，智慧地回應這項要求：

1. **尋找**：透過向量搜尋，尋找*說明*在語意上符合使用者要求的產品，而不只是簡單的關鍵字搜尋。
 2. **說明**：使用 LLM 分析最符合條件的結果，並說明*原因*，建立顧客信任感。
 3. **關聯**：使用圖形查詢找出消費者經常與該推薦產品*一起*購買的其他產品。
-

2. 事前準備

1. **建立雲端專案**：在 Google Cloud 控制台的專案選取器頁面中，選取或建立 Google Cloud 雲端專案。
2. **啟用帳單**：確認 Cloud 專案已啟用帳單。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. **啟用 Cloud Shell**：按一下控制台中的「啟用 Cloud Shell」按鈕，啟用 Cloud Shell。您可以在 Cloud Shell 終端機和編輯器之間切換。



4. **授權及設定專案**：連至 Cloud Shell 後，請確認您已通過驗證，且專案已設為您的專案 ID。

〇〇〇

```
gcloud auth list
gcloud config list project
```

5. 如果未設定專案，請使用下列指令設定，並將 `<PROJECT_ID>` 換成實際專案 ID：

〇〇〇

```
export PROJECT_ID=<PROJECT_ID>
gcloud config set project $PROJECT_ID
```

6. **啟用必要 API**：啟用 Spanner、Vertex AI 和 Compute Engine API。這可能需要幾分鐘的時間。

〇〇〇

```
gcloud services enable \
  spanner.googleapis.com \
  aiplatform.googleapis.com \
  compute.googleapis.com
```

7. **設定幾個會重複使用的環境變數**。

〇〇〇

```
export INSTANCE_ID=my-first-spanner
export INSTANCE_CONFIG=regional-us-central1
```

8. 如果您還沒有 Spanner 執行個體，請[建立免費試用 Spanner 執行個體](#)。您需要 Spanner 執行個體來代管資料庫。我們將使用 `regional-us-central1` 做為設定。如要更新，請前往該頁面。

〇〇〇

```
gcloud spanner instances create $INSTANCE_ID \
  --instance-type=free-instance --config=$INSTANCE_CONFIG \
  --description="Trial Instance"
```

步驟 3 · 約 0 分鐘

3. 架構總覽

Spanner 會封裝所有必要功能，但模型除外，因為模型是代管在 Vertex AI 上。

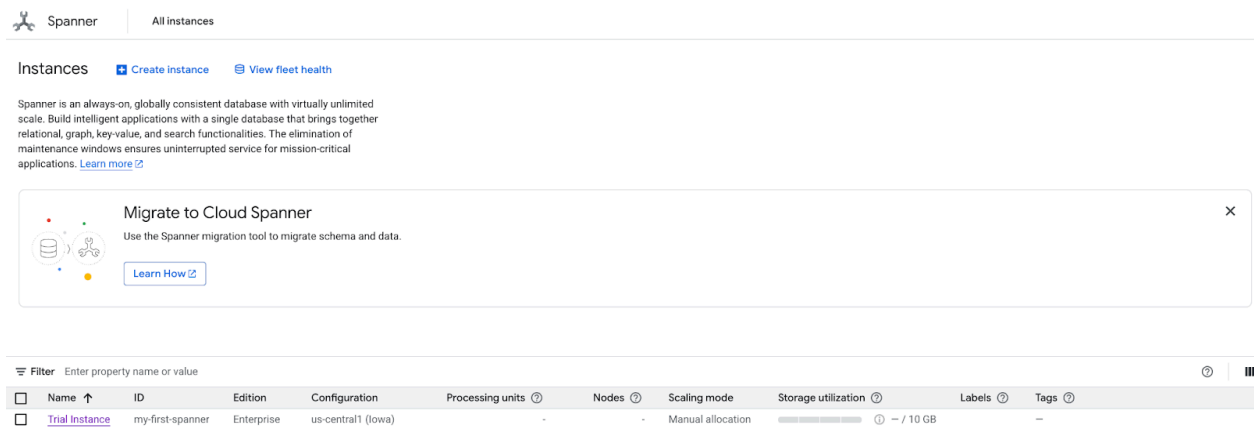
步驟 4 · 約 0 分鐘

4. 步驟 1：設定資料庫並提交第一項查詢。

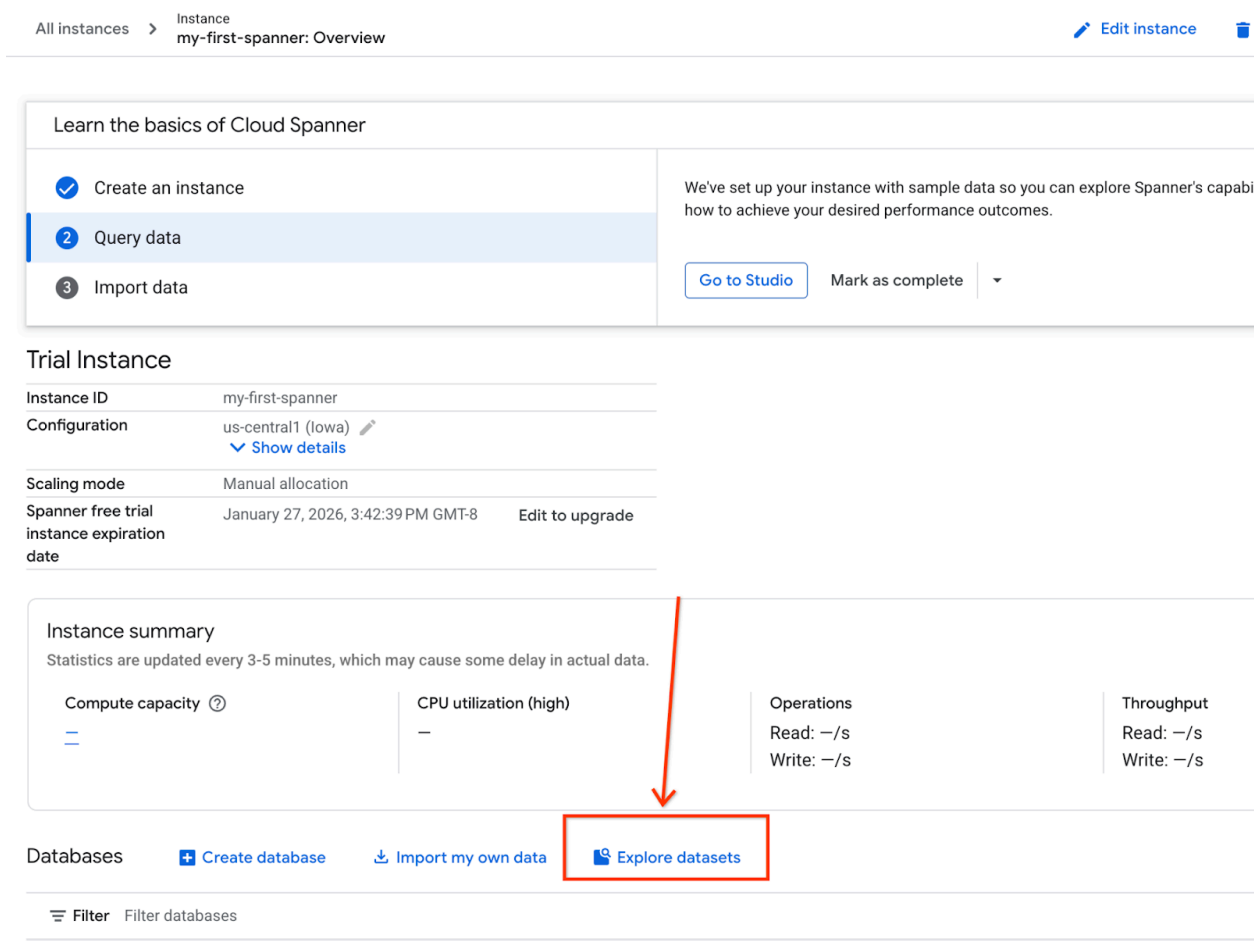
首先，我們需要建立資料庫、載入零售業範例資料，並告知 Spanner 如何與 Vertex AI 通訊。

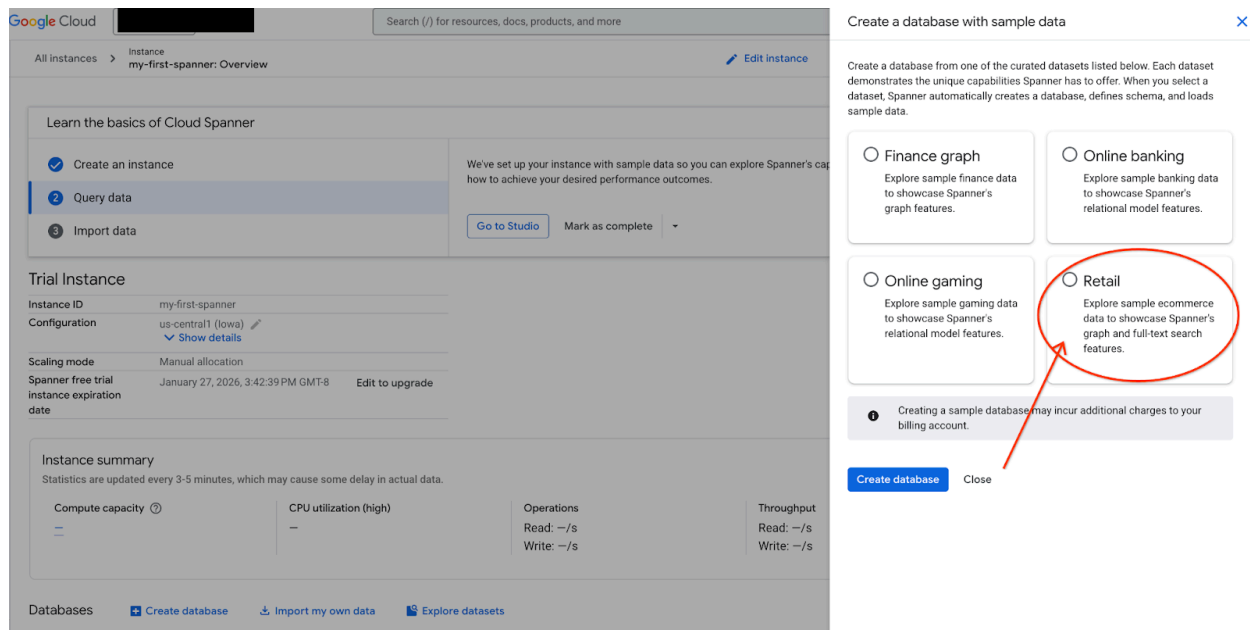
這個部分會用到下列 SQL 指令碼。

- 1. 前往 **Spanner** 的產品頁面。
- 2. 選取正確的例項。

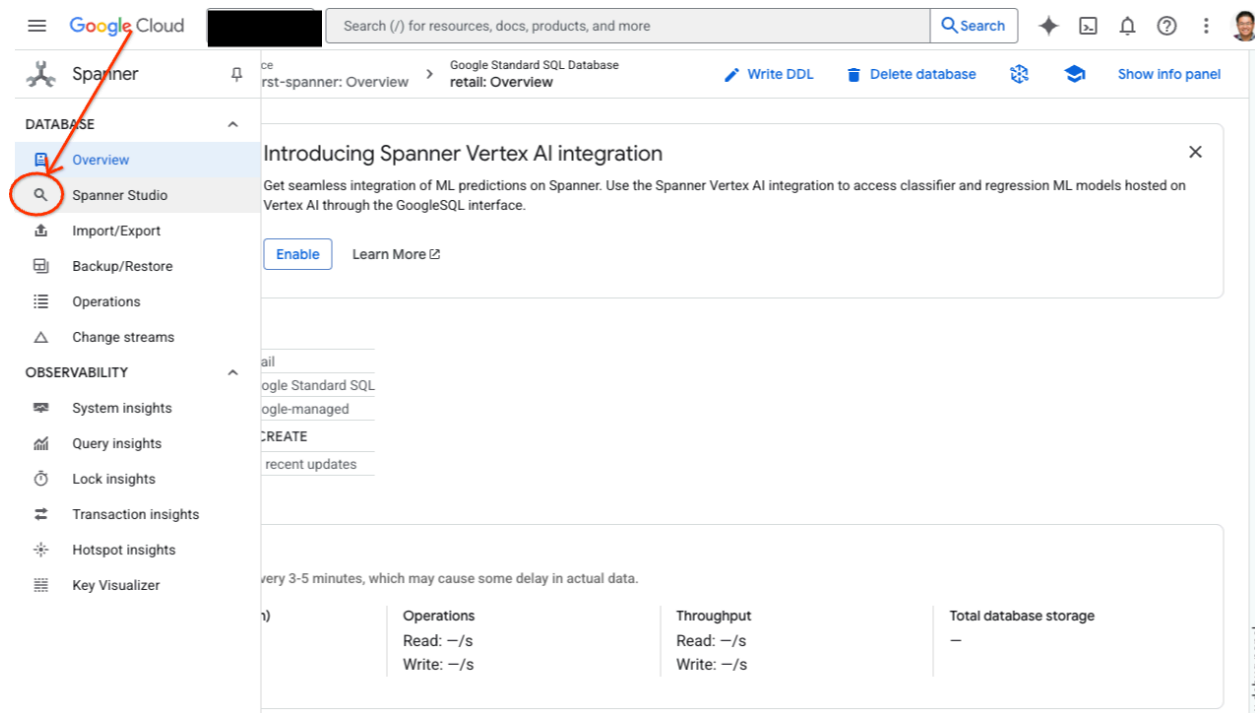


- 3. 在畫面上選取「探索資料集」。然後在彈出式視窗中選取「零售」選項。





4. 前往 **Spanner Studio**。Spanner Studio 包含「Explorer」窗格，可與查詢編輯器和 SQL 查詢結果表整合。您可以在這個介面執行 DDL、DML 和 SQL 陳述式。你必須展開側邊選單，然後尋找放大鏡圖示。



5. 閱讀「產品」表格。建立新分頁，或使用已建立的「未命名的查詢」分頁。

All instances > Instance my-first-spanner: Overview > Google Standard SQL Database retail: Spanner Studio

Open in Datap

Explorer

Search

Schemas 3

Default

Tables 7

Addresses

OrderItems

Orders

Payments

Products

ShoppingCarts

Users

Change streams 0

Views 0

Models 0

Graphs 1

INFORMATION_SCHEMA

SPANNER_SYS

Roles 3

Placements 1

Queries 14

Preview

01. Product Search by Name

Untitled query X

+

Open CLI

Welcome to Spanner Studio!

Use Spanner Studio to see, query, and modify your database objects all in one place. Run queries, create views, and modify schemas. [Learn More](#)

Quick Tour

New SQL Editor Tab

Schema features



Gemini Code Assist: Natural language queries and query prediction

SQL generation suggests a query in response to a natural language question. Just press 'tab' to accept. [Learn More](#)



Execute batch queries offline with Data Boost

With Data Boost, run federated queries without copying data to BigQuery, using independent compute capacity to avoid impacting your existing workloads. [Learn More](#)



Control access to your data

Fine-grained access control combines the benefits of Identity and Access Management (IAM) with traditional SQL role-based access. [Learn More](#)

☐ Skip home tab on startup

```
SELECT *  
FROM Products;
```


5. 步驟 2：建立 AI 模型。

現在，我們來使用 Spanner 物件建立遠端模型。這些 SQL 陳述式會建立連結至 Vertex AI 端點的 Spanner 物件。

1. 在 Spanner Studio 中開啟新分頁，然後建立兩個模型。第一個是 EmbeddingsModel，可讓您生成嵌入。第二個是 LLMModel，可讓您與 LLM 互動 (在本例中為 gemini-2.5-flash)。請確認您已將 <PROJECT_ID> 更新為專案 ID。

```
### Create the Embedding Model object in Spanner
CREATE MODEL EmbeddingsModel INPUT(
  content STRING(MAX),
) OUTPUT(
  embeddings STRUCT<statistics STRUCT<truncated BOOL, token_count FLOAT32>, values
  ARRAY<FLOAT32>>,
) REMOTE OPTIONS (
  endpoint = '//aiplatform.googleapis.com/projects/<PROJECT_ID>/locations/us-
  central1/publishers/google/models/text-embedding-005'
);

### Create the LLM Model object in Spanner
CREATE MODEL LLMModel INPUT(
  prompt STRING(MAX),
) OUTPUT(
  content STRING(MAX),
) REMOTE OPTIONS (
  endpoint = '//aiplatform.googleapis.com/projects/<PROJECT_ID>/locations/us-
  central1/publishers/google/models/gemini-2.5-flash',
  default_batch_size = 1
);
```

2. 注意：請記得將 PROJECT_ID 替換為實際的 \$PROJECT_ID。

The screenshot displays the Google Cloud Spanner Studio interface. The top navigation bar shows 'All instances' > 'Instance my-first-spanner: Overview' > 'Google Standard SQL Database retail: Spanner Studio'. The left sidebar contains an 'Explorer' panel with a search bar and a tree view of the database schema, including Schemas, Tables, Views, Models, and Roles. The main editor area shows a SQL query with two parts: creating an Embedding Model and an LLM Model. The query is as follows:

```
1 ## Create the Embedding Model object in Spanner
2 CREATE MODEL EmbeddingsModel INPUT(
3   content STRING(MAX),
4 ) OUTPUT(
5   embeddings STRUCT<statistics STRUCT<truncated BOOL, token_count FLOAT32>, values ARRAY<FLOAT32>>,
6 ) REMOTE OPTIONS (
7   endpoint = '//aiplatform.googleapis.com/projects/[REDACTED]/locations/us-central1/publishers/google/models/gemini-embedding-001'
8 );
9
10 ## Create the LLM Model object in Spanner
11 CREATE MODEL LLMModel INPUT(
12   prompt STRING(MAX),
13 ) OUTPUT(
14   content STRING(MAX),
15 ) REMOTE OPTIONS (
16   endpoint = '//aiplatform.googleapis.com/projects/[REDACTED]/locations/us-central1/publishers/google/models/gemini-2.5-flash',
17   default_batch_size = 1
18 );
```

Two red arrows point to the '[REDACTED]' placeholders in the endpoint URLs, with the text 'your project ID' written between them. The bottom panel shows the 'Results' tab with a message: 'Update completed. To view more details, go to [Operations Page](#). To view all updates go to [Logging](#).'

測試這個步驟：您可以在 SQL 編輯器中執行下列指令，確認模型是否已建立。

```
SELECT *
FROM information_schema.models;
```

 Run selected 

Save  Format Clear Documentation 

 Valid

```
1 SELECT *
2 FROM information_schema.models;
```

Results Explanation 

 Export  

MODEL_CATALOG	MODEL_SCHEMA	MODEL_NAME	IS_REMOTE
""	""	"EmbeddingsModel"	true
""	""	"LLMModel"	true

Rows per page: 30  1 – 2 of 2  

6. 步驟 3：產生及儲存向量嵌入

我們的「產品」資料表含有文字說明，但 AI 模型會解讀*向量* (數字陣列)。我們需要新增資料欄來儲存這些向量，然後透過 EmbeddingsModel 執行所有產品說明，填入資料欄。

1. **建立新資料表來支援嵌入項目。**首先，請建立可支援嵌入項目的資料表。我們使用的嵌入模型與產品表格範例嵌入不同。您必須確保向量搜尋功能正常運作，且嵌入是從同一個模型生成。

```
CREATE TABLE products_with_embeddings (  
  ProductID INT64,  
  embedding_vector ARRAY<FLOAT32>(vector_length=>768),  
  embedding_text STRING(MAX)  
)  
PRIMARY KEY (ProductID);
```

2. **在新的資料表中填入模型生成的嵌入項目。**為求簡單，我們在這裡使用 [insert into 陳述式](#)。這會將查詢結果推送到您剛才建立的資料表。

SQL 陳述式會先擷取並串連所有要產生嵌入的相關文字資料欄。然後傳回相關資訊，包括我們使用的文字。一般來說，這並非必要步驟，但我們仍會加入，方便您查看結果。

```
INSERT INTO products_with_embeddings (productId, embedding_text, embedding_vector)  
SELECT  
  ProductID,  
  content as embedding_text,  
  embeddings.values as embedding_vector  
FROM ML.PREDICT(  
  MODEL EmbeddingsModel,  
  (  
    SELECT  
      ProductID,  
      embedding_text AS content  
    FROM (  
      SELECT  
        ProductID,  
        CONCAT(  
          Category,  
          " ",  
          Description,  
          " ",  
          Name  
        ) AS embedding_text  
      FROM products))));
```

3. **檢查新的嵌入內容。**現在應該會看到產生的嵌入內容。

```
SELECT *  
FROM products_with_embeddings  
LIMIT 1;
```

All instances > Instance my-first-spanner: Overview > Google Standard SQL Database retail: Spanner Studio

Open in Dataplex

Learn

Explorer

Orders

Payments

Products

Columns 9

ProductID

Tr Name

Tr Description

Tr ImageURL

PriceUSD

Tr Category

ProductEmbed...

Name_Tokens

Description_Tokens

Keys 1

ProductID

Indexes 3

products_with_embeddings

ShoppingCarts

Users

Run selected

Save

Format

Clear

Documentation

Valid

1 SELECT *

2 FROM products_with_embeddings

3 LIMIT 1;

Results

Explanation

Export

ProductID	embedding_vector	embedding_text
123	[0.0018158768070861697, -0.0022627890575677156, 0.02420058101415634,	"Electronics Premium smartphone with advanced features and long battery life. High-

Rows per page: 30 1 - 1 of 1 < >

7. 步驟 4：建立 ANN 搜尋的向量索引

如要即時搜尋數百萬個向量，我們需要索引。這個索引可啟用近似最近鄰居 (ANN) 搜尋，速度極快且可水平擴展。

重要事項：

建立向量索引需要大量運算資源。如果資料集很大，可能需要 10 分鐘以上。

1. 執行下列 DDL 查詢來建立索引。我們將 `COSINE` 指定為距離指標，這非常適合語意文字搜尋。請注意，WHERE 子句實際上是必要的，因為 Spanner 會將其設為查詢的必要條件。

```
CREATE VECTOR INDEX DescriptionEmbeddingIndex
ON products_with_embeddings(embedding_vector)
WHERE embedding_vector IS NOT NULL
OPTIONS (
  distance_type = 'COSINE'
);
```

2. 在「作業」分頁中查看索引建立狀態。

Spanner

DATABASE

Overview

Spanner Studio

Import/Export

Backup/Restore

Operations

Change streams

OBSERVABILITY

System insights

Query insights

Lock insights

Transaction insights

Hotspot insights

Key Visualizer

ce

rst-spanner: Overview

Google Standard SQL Database

retail: Operations

Operations for this database. An operation is viewable from the time it starts to seven days after it ends. [Logging](#) to see a full history of your activity, including backup deletions.

	Start time	End time	
Indexing	Oct 30, 4:47 PM UTC-7	0%	Details
Vector index	Oct 30, 4:46 PM UTC-7	Oct 30, 4:46 PM UTC-7	Details
Embeddings	Oct 30, 4:29 PM UTC-7	Oct 30, 4:29 PM UTC-7	Details
Embeddings	Oct 30, 4:28 PM UTC-7	Oct 30, 4:28 PM UTC-7	Details
Embeddings	Oct 30, 4:28 PM UTC-7	Oct 30, 4:28 PM UTC-7	Details
Embeddings	Oct 30, 4:28 PM UTC-7	Oct 30, 4:28 PM UTC-7	Details
	Oct 30,	Oct 30, 4:26 PM UTC-7	Details

8. 步驟 5：使用 K 近鄰 (KNN) 搜尋功能尋找建議

現在來到有趣的部分！讓我們找出符合顧客查詢的產品：「我想購買高效能鍵盤。我偶爾會在海灘上寫程式，所以可能會弄濕。」

我們將從 **K-Nearest Neighbor (KNN)** 搜尋開始。這是**精確**搜尋，會將查詢向量與**每個**產品向量進行比較。這種方法很精確，但處理大型資料集時可能會比較慢 (這也是我們在步驟 5 中建立 ANN 索引的原因)。

這項查詢會執行兩項作業：

1. 子查詢會使用 ML.PREDICT 取得顧客查詢的嵌入向量。
2. 外部查詢會使用 COSINE_DISTANCE 計算查詢向量與每個產品的 embedding_vector 之間的「距離」。距離越小，表示越相符。

```
SELECT
  productid,
  embedding_text,
  COSINE_DISTANCE(
    embedding_vector,
    (
      SELECT embeddings.values
      FROM ML.PREDICT(
        MODEL EmbeddingsModel,
        (SELECT "I'd like to buy a high performance keyboard. I sometimes code while I'm at
the beach so it may get wet." AS content)
      )
    )
  ) AS distance
FROM products_with_embeddings
WHERE embedding_vector IS NOT NULL
ORDER BY distance
LIMIT 5;
```

畫面上會顯示產品清單，抗水鍵盤會列在最上方。

9. 步驟 6：使用近似 (ANN) 搜尋功能尋找建議

KNN 很棒，但對於每秒有數百萬項產品和數千項每秒查詢次數的正式環境系統，我們需要 ANN 索引的速度。

使用索引時，您必須指定 APPROX_COSINE_DISTANCE 函式。

1. 如上所述，取得文字的向量嵌入。我們將該結果與 products_with_embeddings 資料表中的記錄交叉聯結，方便您在 APPROX_COSINE_DISTANCE 函式中使用。

```
WITH vector_query as
(
  SELECT embeddings.values as vector
  FROM ML.PREDICT(
    MODEL EmbeddingsModel,
    (SELECT "I'd like to buy a high performance keyboard. I sometimes code while I'm at the
beach so it may get wet." as content)
  )
)
SELECT
ProductID,
embedding_text,
APPROX_COSINE_DISTANCE(embedding_vector, vector, options => JSON '{"num_leaves_to_search":
10}') distance
FROM products_with_embeddings @{{force_index=DescriptionEmbeddingIndex}},
vector_query
WHERE embedding_vector IS NOT NULL
ORDER BY distance
LIMIT 5;
```

預期輸出內容：結果應與 KNN 查詢相同或非常相似，但使用索引後，執行效率會大幅提升。您可能不會在範例中注意到這一點。

10. 步驟 7：使用 LLM 解釋建議

只顯示產品清單是不錯的做法，但說明產品是否合適的原因更好。我們可以透過 LLModel (Gemini) 執行這項操作。

這項查詢會將步驟 4 中的 KNN 查詢，巢狀內嵌至 ML.PREDICT 呼叫中。我們使用 CONCAT 為 LLM 建構提示，提供下列資訊：

1. 明確的指示 (例如「請回答『是』或『否』，並說明原因...」)。
2. 顧客的原始查詢。
3. 每個最相符產品的名稱和說明。

接著，LLM 會根據查詢評估各項產品，並以自然語言提供回覆。

```
SELECT
  ProductID,
  embedding_text,
  content AS LLMResponse
FROM ML.PREDICT(
  MODEL LLModel,
  (
    SELECT
      ProductID,
      embedding_text,
      CONCAT(
        "Answer with 'Yes' or 'No' and explain why: Is this a good fit for me?",
        "I'd like to buy a high performance keyboard. I sometimes code while I'm at the
beach so it may get wet. \n",
        "Product Description:", embedding_text
      ) AS prompt,
    FROM products_with_embeddings
    WHERE embedding_vector IS NOT NULL
    ORDER BY COSINE_DISTANCE(
      embedding_vector,
      (
        SELECT embeddings.values
        FROM ML.PREDICT(
          MODEL EmbeddingsModel,
          (SELECT "I'd like to buy a high performance keyboard. I sometimes code while I'm
at the beach so it may get wet." AS content)
        )
      )
    )
  ) AS prompt,
  FROM products_with_embeddings
  WHERE embedding_vector IS NOT NULL
  ORDER BY COSINE_DISTANCE(
    embedding_vector,
    (
      SELECT embeddings.values
      FROM ML.PREDICT(
        MODEL EmbeddingsModel,
        (SELECT "I'd like to buy a high performance keyboard. I sometimes code while I'm
at the beach so it may get wet." AS content)
      )
    )
  )
  LIMIT 5
),
STRUCT(1056 AS maxOutputTokens)
);
```

預期輸出內容：您會看到含有新 LLMResponse 資料欄的表格。回覆內容應類似這樣：「否。原因如下：* 『抗水』並非『防水』。「抗水」鍵盤可承受潑濺、小雨或溢出

11. 步驟 8：建立屬性圖

現在來看看另一種建議：「購買這項產品的顧客也買了...」

這是以關係為基礎的查詢。屬性圖是處理這項作業的絕佳工具。Spanner 可讓您在現有資料表上建立圖表，不必複製資料。

這個 DDL 陳述式會定義圖表：

- **節點：** `Product` 和 `User` 資料表。節點是您想從中衍生關係的實體，您想瞭解購買您產品的顧客是否也購買了「XYZ」產品。
- **邊緣：** `Orders` 表格，將 `User` (來源) 連結至 `Product` (目的地)，並加上「已購買」標籤。邊緣則提供使用者與所購商品之間的關係。

```
CREATE PROPERTY GRAPH RetailGraph
  NODE TABLES (
    products_with_embeddings,
    Orders
  )
  EDGE TABLES (
    OrderItems
      SOURCE KEY (OrderID) REFERENCES Orders
      DESTINATION KEY (ProductID) REFERENCES products_with_embeddings
      LABEL Purchased
  );
```

12. 步驟 9：結合向量搜尋和圖形查詢

這是最有效的步驟。我們會在單一陳述式中結合 AI 向量搜尋和圖形查詢，找出相關產品。

這項查詢分為三部分，並以 `NEXT statement` 分隔，讓我們將其細分為幾個區段。

1. 首先，我們會使用向量搜尋找出最相符的結果。
2. `ML.PREDICT` 會使用 `EmbeddingsModel`，從使用者的文字查詢生成向量嵌入。
3. 這項查詢會計算所有產品的新嵌入向量與儲存的 `p.embedding_vector` 之間的 `COSINE_DISTANCE`。
4. 系統會選取並傳回距離最短 (語意相似度最高) 的單一最佳比對產品。
5. 接著，我們會在圖表中搜尋關係。

```
NEXT MATCH (bestMatch)<-[:Purchased]-(user:Orders)-[:Purchased]->
(purchasedWith:products_with_embeddings)
```

1. 查詢會從 `bestMatch` 回溯至常見的 `Orders` 節點 (使用者)，然後轉送至其他 `purchasedWith` 產品。
2. 這會篩除原始產品，並使用 `GROUP BY` 和 `COUNT(1)` 匯總項目共同購買的頻率。
3. 系統會傳回前 3 項共同購買的產品 (`purchasedWith`)，並依共同出現的頻率排序。

此外，我們還會找出使用者與訂單的關係。

```
NEXT MATCH (bestMatch)<-[:Purchased]-(user:Orders)-[purchased:Purchased]->(purchasedWith)
```

1. 這個中繼步驟會執行周遊模式，以繫結主要實體：`bestMatch`、連結的使用者：`Orders` 節點，以及 `purchasedWith` 項目。
2. 具體來說，這會將關係本身繫結為已購買，以便在下一個步驟中擷取資料。
3. 這個模式可確保建立脈絡，以便擷取訂單和產品的詳細資料。
4. 最後，我們輸出要傳回的結果，因為圖形節點必須先經過格式化，才能以 `SQL` 結果的形式傳回。

```

GRAPH RetailGraph
MATCH (p:products_with_embeddings)
WHERE p.embedding_vector IS NOT NULL
RETURN p AS bestMatch
ORDER BY COSINE_DISTANCE(
  p.embedding_vector,
  (
    SELECT embeddings.values
    FROM ML.PREDICT(
      MODEL EmbeddingsModel,
      (SELECT "I'd like to buy a high performance keyboard. I sometimes code while I'm at the
beach so it may get wet." AS content)
    )
  )
)
LIMIT 1

NEXT
MATCH (bestMatch)<-[:Purchased]-(user:Orders)-[:Purchased]->
(purchasedWith:products_with_embeddings)
FILTER bestMatch.productId <> purchasedWith.productId
RETURN bestMatch, purchasedWith
GROUP BY bestMatch, purchasedWith
ORDER BY COUNT(1) DESC
LIMIT 3

NEXT
MATCH (bestMatch)<-[:Purchased]-(user:Orders)-[purchased:Purchased]->(purchasedWith)
RETURN
  TO_JSON(Purchased) AS purchased,
  TO_JSON(user.OrderID) AS user,
  TO_JSON(purchasedWith.productId) AS purchasedWith;

```

預期輸出內容：您會看到代表前 3 項共同購買商品的 JSON 物件，提供交叉銷售建議。

13. 正在清除所用資源

為避免產生費用，您可以刪除自己建立的資源。

1. 刪除 **Spanner** 執行個體：刪除執行個體時，資料庫也會一併刪除。

```
gcloud spanner instances delete my-first-spanner --quiet
```

2. 刪除 **Google Cloud** 專案：如果您是專為本程式碼研究室建立專案，刪除專案就是最簡單的清理方式。

- 前往 Google Cloud 控制台的「管理資源」頁面。
- 選取專案並按一下「刪除」。

 **恭喜！**

您已成功使用 Spanner AI 和 Graph 建構精密的即時推薦系統！

您已瞭解如何將 Spanner 與 Vertex AI 整合，用於嵌入和 LLM 生成作業；如何執行高速向量搜尋 (KNN 和 ANN) 來尋找語意相關的產品；以及如何使用圖形查詢來發掘產品關係。您已建構的系統不僅能尋找產品，還能說明推薦內容和建議相關項目，而且全都在單一可擴充的資料庫中完成。
